

BUSINESS REQUIREMENTS DOCUMENT

Peter Lynch IT Stock Research Agent

Multi-Agent System using Google ADK + MCP + IndianAPI

Project Name	IT Lynch Agent — Prototype v1.0
Version	1.0 — Initial Prototype
Deadline	2 days from kick-off
Author	Vivek
Date	05 July 2026
Status	DRAFT — For Development

Purpose: Prototype demonstrating Google Agent Development Kit (ADK) + MCP integration for quantitative equity research

1. Executive Summary

This document specifies the requirements for a prototype multi-agent equity research system that applies Peter Lynch's quantitative investment framework to Indian IT stocks. The system consists of three co-operating agents built using Google's Agent Development Kit (ADK) with Gemini as the reasoning engine, demonstrating ADK multi-agent orchestration and MCP (Model Context Protocol) tool integration.

Submission goal: A working prototype delivered within 2 days that serves as a project artifact demonstrating proficiency in Google ADK agent design, MCP tool integration, and LLM-driven equity analysis pipelines.

1.1 What the System Does

- Agent 1 pulls live financial data from IndianAPI and evaluates each IT stock against Lynch's quantitative thresholds — PEG ratio, debt/equity, net cash, FCF conversion, revenue growth, and PE vs historical range
- Agent 2 pulls IT-sector KPI data — EBIT margin trend, analyst consensus, EPS revision direction, ROE trend, and revenue growth trajectory — and scores each stock on operational quality
- Agent 3 consumes scored outputs from both agents, ranks all stocks on a 100-point composite Lynch Score, and generates a standalone HTML report with rankings, metric breakdowns, and buy/hold/avoid verdicts

1.2 Why Google ADK + MCP

- Google ADK provides a structured agent runtime: each agent is an ADK LlmAgent with its own instruction, model (Gemini 2.0 Flash), and tool set
- MCP integration allows IndianAPI calls to be wrapped as MCP tools — demonstrating MCP as a composable tool-provision protocol
- Agent 3 calling Agents 1 and 2 as sub-agents showcases ADK's multi-agent delegation capability

2. Project Scope

2.1 In Scope — Prototype v1.0

In Scope Item	Detail
Stock universe	8 Indian IT stocks: TCS, Infosys, HCL Tech, Wipro, Persistent, Coforge, Mphasis, LTT
Agent 1	Lynch quantitative scoring using IndianAPI live data
Agent 2	IT KPI scoring using IndianAPI /historical_stats, /stock_forecasts, /stock
Agent 3	Composite ranking engine + standalone HTML report generation
Data sources	IndianAPI (primary), yfinance (USD/INR FX rate, fallback price history)
ADK integration	3 ADK Agents, tools as Python @tool functions / MCP tools, sequential pipeline
Output	One standalone HTML file per run — rankings, scores, charts, verdicts
Run mode	CLI: python run_agents.py — executes all 3 agents sequentially

2.2 Explicitly Out of Scope — Phase 2

Out of Scope (Phase 2)	Reason
Concall transcript parsing	IndianAPI has no transcript endpoint; requires BSE PDF scraping + LLM extraction
TCV / attrition / utilisation	Only available from earnings call PDFs — Phase 2 BSE filing poller
Real-time intraday data	Prototype uses end-of-day / quarterly data only
Scheduled runs	Prototype is manually triggered; cron scheduling is Phase 2
Other sectors (Pharma, Financials)	IT sector only; extensible architecture planned

3. System Architecture

3.1 High-Level Pipeline

```
run_agents.py (CLI entry point)
|
+--> Agent 1: Lynch Quantitative Analyser
| Tools: IndianAPI /stock, /historical_data, /historical_stats, /stock_target_price
| Output: data/scored_quant.json
|
+--> Agent 2: IT KPI Analyser
| Tools: IndianAPI /stock_forecasts, /historical_stats, /stock
| Output: data/scored_kpi.json
|
+--> Agent 3: Ranking Engine + Report Generator
Tools: read_scored_json, write_html_report
Output: output/report_YYYYMMDD_HHMMSS.html
```

3.2 Google ADK Agent Model

Each of the three agents is defined as a `google.adk.agents.LlmAgent` instance with:

- `name` — unique agent identifier
- `model` — `gemini-2.0-flash` (fast, low cost, sufficient for structured analysis)
- `instruction` — system prompt defining the agent's role, data sources, and output format
- `tools` — list of `@tool`-decorated Python functions or MCP-provided tools
- `output_key` — session state key where the agent writes its JSON output

3.3 MCP Integration Points

MCP / Python Tool	Wraps	Used by Agent
<code>get_stock_snapshot(name)</code>	IndianAPI /stock	Agent 1 + Agent 2
<code>get_historical_stats(name, stats)</code>	IndianAPI /historical_stats	Agent 1 + Agent 2
<code>get_historical_data(name, filter)</code>	IndianAPI /historical_data	Agent 1
<code>get_analyst_targets(stock_id)</code>	IndianAPI /stock_target_price	Agent 2
<code>get_forecasts(stock_id, measure)</code>	IndianAPI /stock_forecasts	Agent 2
<code>get_fx_rate()</code>	yfinance USDINR=X	Agent 2 (FX context)
<code>read_scored_json(agent_id)</code>	Local filesystem read	Agent 3
<code>write_html_report(content)</code>	Local filesystem write	Agent 3

MCP implementation note: For the prototype, each tool is a Python function decorated with `@tool` from `google.adk.tools`. In production these would be registered as an MCP server, exposing IndianAPI as a reusable endpoint callable by any ADK agent.

4. Agent 1 — Lynch Quantitative Analyser

Agent Name	lynch_quant_agent
Model	gemini-2.0-flash
Role	Fetches financial data and scores each stock against Lynch quantitative thresholds
Input	Stock universe list: TCS, INFY, HCLTECH, WIPRO, PERSISTENT, COFORGE, MPHASIS, LTTS
Output	data/scored_quant.json — array of StockQuantScore objects
Tools	get_stock_snapshot, get_historical_stats, get_historical_data

4.1 Lynch Quantitative Scoring — 60 Points Total

Metric	Data Source	Threshold / Formula	Max Pts	Scoring
PEG Ratio	/stock keyMetrics	P/E / 5yr EPS growth	15	<=0.5=15 0.5-1.0=12 1.0-1.5=8 1.5-2.0=4 >2.0=0
Debt / Equity	/historical_stats ratios	Long-term debt / equity	12	<0.3=12 0.3-0.5=9 0.5-0.8=5 >0.8=0
Revenue CAGR (5yr)	/historical_stats yoy_results	5yr revenue CAGR %	12	>20%=12 15-20%=9 10-15%=6 5-10%=3 <5%=0
P/E vs 5yr Median	/historical_data filter=pe	Current P/E / 5yr median	10	<0.85=10 0.85-1.0=8 1.0-1.15=5 1.15-1.3=2 >1.3=0
Net Cash % Mkt Cap	/stock keyMetrics	(Cash - Debt) / Mkt Cap	6	>20%=6 10-20%=4 0-10%=2 Net debt=0
FCF Conversion	/historical_stats cashflow	OCF / Net Profit	5	>85%=5 70-85%=4 55-70%=2 <55%=0

4.2 Agent Instruction (System Prompt)

You are a Peter Lynch quantitative analyst specialising in Indian IT stocks.
For each stock: call get_stock_snapshot, get_historical_stats (ratios, yoy_results, cashflow), and get_historical_data (pe). Apply the scoring rubric exactly and compute quant_score/60.
Return a JSON array conforming to StockQuantScore schema — JSON only, no prose.

4.3 Output Schema — scored_quant.json (key fields)

```
{ "symbol": "TCS", "current_price": 3580.50, "pe_current": 27.4,
  "pe_5yr_median": 28.1, "peg_ratio": 1.37, "debt_equity": 0.08,
  "revenue_cagr_5yr": 11.2, "net_cash_pct_mcap": 14.5, "fcf_conversion": 91.2,
  "scores": { "peg": 8, "debt_equity": 12, "revenue_growth": 6,
    "pe_vs_hist": 8, "net_cash": 4, "fcf": 5 },
  "quant_score": 43, "lynch_category": "Stalwart" }
```

5. Agent 2 — IT KPI Analyser

Agent Name	it_kpi_agent
Model	gemini-2.0-flash
Role	Analyses IT operational quality: margin trends, analyst consensus, EPS revisions, ROE trajectory
Input	Same stock universe list as Agent 1
Output	data/scored_kpi.json — array of StockKPIScore objects
Tools	get_stock_snapshot, get_historical_stats, get_analyst_targets, get_forecasts, get_fx_rate

Note on IT KPI data availability: TCV, attrition rate, and utilisation rate are only available in quarterly concall transcripts, which IndianAPI does not provide. These are Phase 2 features. Agent 2 uses available IndianAPI endpoints as proxies.

5.1 IT KPI Scoring — 40 Points Total

KPI Metric	Data Source	Proxies For	Max Pts	Scoring Rule
EBIT Margin Trend (4Q)	/historical_stats/quarter_results	Operational efficiency	12	Expanding >150bps=12 Stable=8 Declining=4
Revenue Growth Trend (4Q)	/historical_stats/quarter_results	Demand acceleration	10	Accel 2+ qtrs=10 Stable=6 Decel 1qtr=3 Dec=0
Analyst Consensus	/stock_target_price	Institutional view	8	>=70% buy=8 50-70%=6 30-50%=3 Sell=0
ROE Trend (3yr)	/historical_stats/ratios	Capital efficiency	6	>20% improving=6 >20% stable=4 15-20%=2
EPS Estimate Direction	/stock_forecasts/EPS	Earnings momentum	4	Upward revision=4 No change=2 Downward=0

5.2 Phase 2 KPIs — Not in Prototype

KPI	Phase 2 Source	Why Deferred
Deal TCV (USD mn)	Concall PDF + LLM extraction	IndianAPI has no transcript endpoint
Attrition Rate (LTM %)	Concall PDF + LLM extraction	Only disclosed in earnings call Q&A
Utilisation Rate (%)	Concall PDF + LLM extraction	Only disclosed in earnings call commentary
Revenue per Employee	IndianAPI + BSE headcount filing	Headcount only in annual reports

5.3 Output Schema — scored_kpi.json (key fields)

```
{ "symbol": "TCS", "ebit_margin_trend": "expanding",
  "revenue_trend": "stable", "analyst_buy_pct": 72.0,
  "analyst_consensus": "StrongBuy", "roe_trend": "improving",
  "eps_revision": "upward",
  "scores": { "ebit_margin": 12, "revenue_trend": 6,
    "analyst_consensus": 8, "roe_trend": 6, "eps_revision": 4 },
  "kpi_score": 36, "kpi_score_pct": 90.0 }
```

6. Agent 3 — Ranking Engine & HTML Report Generator

Agent Name	report_agent
Model	gemini-2.0-flash
Role	Merges Agent 1+2 outputs, computes composite Lynch Score, ranks stocks, generates HTML report
Input	data/scored_quant.json + data/scored_kpi.json
Output	output/report_YYYYMMDD_HHMMSS.html — single self-contained HTML file
Tools	read_scored_json, write_html_report

6.1 Composite Lynch Score

Lynch Score = quant_score (60 pts) + kpi_score (40 pts) = 100 pts total

Verdict mapping:

80 - 100 => STRONG BUY (Lynch would be excited about this stock)

65 - 79 => BUY (Passes most filters, good risk/reward)

50 - 64 => WATCH (Interesting but wait for better entry)

35 - 49 => NEUTRAL (Does not clear Lynch's bar currently)

0 - 34 => AVOID (Multiple Lynch red flags present)

6.2 HTML Report Structure

Section	Content
Header	Title, run timestamp, stock universe, data sources (IndianAPI + yfinance)
Executive Summary	Top 3 stocks by Lynch Score with one-line buy thesis; overall sector signal
Rankings Table	All 8 stocks ranked by Lynch Score — sortable: score, quant, KPI, PEG, D/E, verdict
Score Bar Charts	Horizontal bar chart per stock (quant vs KPI contribution) via Chart.js CDN
Stock Cards	One card per stock: all raw metrics, score breakdown, Lynch category badge, verdict
Methodology	Collapsible section explaining the rubric — makes report self-contained
Disclaimer	Standard not-investment-advice footer

6.3 HTML Report Design Spec

- Single .html file — all CSS and JavaScript embedded inline; no external dependencies except Chart.js CDN
- Colour coding: Strong Buy = green, Buy = teal, Watch = yellow, Neutral = grey, Avoid = red
- Sortable rankings table using vanilla JavaScript — no frameworks
- Collapsible stock cards — click to expand full metric detail
- Header credits: 'Powered by Google ADK + Gemini' for project artifact context

7. Data Pipeline

7.1 IndianAPI Endpoint Mapping

Endpoint	Parameters	Agent	Fields Consumed
GET /stock	name={symbol}	1 + 2	currentPrice, keyMetrics (PE, PB, ROE), shareholding, recentNews
GET /historical_stats	stats=ratios	1 + 2	D/E, ROE (3yr trend), ROCE, current ratio
GET /historical_stats	stats=yoy_results	1	Annual revenue (5yr) to compute CAGR
GET /historical_stats	stats=cashflow	1	OCF, Net Profit to compute FCF conversion
GET /historical_stats	stats=quarter_results	2	Quarterly Revenue, EBIT (4 quarters)
GET /historical_data	filter=pe	1	P/E time series to compute 5yr median PE
GET /stock_target_price	stock_id={symbol}	2	recommendation breakdown (buy/hold/sell count)
GET /stock_forecasts	measure_code=EPS, age=OneWeekAge	2	EPS estimate direction

7.2 Authentication & Rate Limiting

IndianAPI Key: store in .env as INDIANAPI_KEY — never hardcode in source
All IndianAPI calls include header: X-API-Key: {INDIANAPI_KEY}
Rate limiting: add 0.5 second sleep between sequential IndianAPI calls
yfinance: No authentication required

7.3 Error Handling

- If IndianAPI returns empty {} for a field: log warning, score that metric as 0, do not crash
- If a stock returns 404 or rate-limit error: retry once after 2 seconds, then skip and flag in report
- If PE history has fewer than 8 data points: use available data and note 'Limited history' in report
- All tool call results logged to data/run_log_YYYYMMDD.jsonl for debugging

8. Technology Stack

Component	Technology	Purpose
Agent Framework	Google ADK (google-adk)	Agent definition, tool execution, session management, multi-agent pipeline
LLM	Gemini 2.0 Flash	Reasoning engine — fast, low-cost, structured output capable
Primary Data	IndianAPI (indianapi.in)	Financials, ratios, PE history, analyst targets, EPS forecasts, shareholding
Supplementary	yfinance	USD/INR FX rate, fallback price data
Tool Protocol	MCP (Model Context Protocol)	Tools as @tool functions; conceptually MCP-compatible for future externalisation
Data Storage	Local JSON files (data/)	Intermediate scored outputs between agents; run logs
Report Output	Standalone HTML + Chart.js	Single-file research report with embedded charts
Environment	Python 3.11+, .env for secrets	Local development; virtualenv or uv

8.1 Project Structure

```
lynch_it_agent/  
run_agents.py # CLI entry point  
agents/  
quant_agent.py # Agent 1 (LlmAgent + tools)  
kpi_agent.py # Agent 2 (LlmAgent + tools)  
report_agent.py # Agent 3 (LlmAgent + tools)  
tools/  
indianapi_tools.py # @tool wrappers for IndianAPI  
yfinance_tools.py # @tool wrappers for yfinance  
scoring/  
quant_scorer.py # Lynch quantitative scoring (60 pts)  
kpi_scorer.py # IT KPI scoring (40 pts)  
report/  
html_template.py # HTML report generator  
data/ # Created at runtime  
scored_quant.json # Agent 1 output  
scored_kpi.json # Agent 2 output  
run_log_YYYYMMDD.jsonl # Debug log  
output/ # Created at runtime  
report_YYYYMMDD_HHMMSS.html  
.env # INDIANAPI_KEY, GOOGLE_API_KEY  
requirements.txt
```

8.2 Key Dependencies — requirements.txt

```
google-adk>=0.3.0
google-generativeai>=0.8.0
yfinance>=0.2.40
requests>=2.31.0
python-dotenv>=1.0.0
```

9. Stock Universe

Symbol	Company Name	NSE Code	Lynch Category	Segment
TCS	Tata Consultancy Services	TCS.NS	Stalwart	Large IT
INFY	Infosys Ltd	INFY.NS	Stalwart	Large IT
HCLTECH	HCL Technologies Ltd	HCLTECH.NS	Stalwart	Large IT
WIPRO	Wipro Ltd	WIPRO.NS	Stalwart	Large IT
PERSISTENT	Persistent Systems Ltd	PERSISTENT.NS	Fast Grower	Mid-cap IT
COFORGE	Coforge Ltd	COFORGE.NS	Fast Grower	Mid-cap IT
MPHASIS	Mphasis Ltd	MPHASIS.NS	Stalwart	Mid-cap IT
LTTS	L&T Technology Services	LTTS.NS	Fast Grower	Engineering R&D IT

10. Two-Day Build Plan

Total estimated: 14-16 hours across 2 days. Build and test incrementally — do not run the full pipeline until Day 2 afternoon.

Day 1 — Foundation + Agent 1 (7-8 hours)

Time	Task	Deliverable	Test
0.5h	Project setup: repo, venv/uv, requirements.txt, .env	Repo/API keys created	import google.adk succeeds
1.0h	Build indianapi_tools.py: all 7 @tool functions with tools/indianapi_tools.py	tools/indianapi_tools.py	get_stock_snapshot('TCS') returns shareholding
0.5h	Build yfinance_tools.py: get_fx_rate() tool	tools/yfinance_tools.py	USD/INR rate returned
1.5h	Build scoring/quant_scorer.py: all 6 scoring functions with thresholds	scoring/quant_scorer.py	Unit test with hardcoded TCS data
2.0h	Build agents/quant_agent.py: ADK LlmAgent, instruct quant prompts generated	agents/quant_agent.py	8 stocks, scores 0-60, no nulls
1.0h	Debug IndianAPI response field mapping	quant_agent running cleanly	scored_quant.json passes schema check
0.5h	Commit Day 1, review scores for sanity	Git commit	TCS/Infosys score higher than Wipro

Day 2 — Agent 2 + Agent 3 + Integration (7-8 hours)

Time	Task	Deliverable	Test
1.5h	Build scoring/kpi_scorer.py and agents/kpi_agents.py	scoring/kpi_scorer.py generated	EBIT margin trend classified correctly
0.5h	Debug Agent 2 analyst consensus and EPS revisited	agents/kpi_agents.py	Analyst buy % is float 0-100
2.0h	Build report/html_template.py: rankings table, ChartJS template complete	report/html_template.py	Report opens in browser, charts render
1.0h	Build agents/report_agent.py: merge JSONs, complete HTML	agents/report_agent.py	8 stocks ranked; Persistent/Coforge outrank TCS
0.5h	Build run_agents.py: sequential pipeline runner	run_agents.py	python run_agents.py completes in < 3 min
1.0h	Polish: README.md, Phase 2 roadmap section	README.md, Phase 2 roadmap	Report looks professional
0.5h	Final commit + zip for submission	Project submitted	End-to-end run from clean checkout works

11. Assumptions, Dependencies & Risks

11.1 Assumptions

- IndianAPI account signed up and INDIANAPI_KEY available before coding begins
- /stock endpoint returns actual data (not empty {}) for shareholding and keyMetrics — verify Day 1, hour 1
- Google ADK 0.3+ supports LlmAgent with @tool-decorated Python functions
- Gemini 2.0 Flash produces structured JSON when instructed — use response_schema to enforce

11.2 Key Risks & Mitigations

Risk	Mitigation
IndianAPI returns empty {} on free tier	Test Day 1 hour 1. If empty: use yfinance for financials as fallback. Document the gap.
Google ADK API version changes	Pin exact version in requirements.txt. Check ADK GitHub changelog before starting.
Gemini returns malformed JSON	Use ADK response_schema to enforce JSON schema. Validate before writing scored_quant.json.
IndianAPI rate limits block bulk calls	Add 0.5s sleep between calls. If throttled: reduce universe to 5 stocks for demo.
HTML chart rendering fails offline	Chart.js from CDN. Include fallback text table — rankings table always works without JS.

12. Acceptance Criteria

The prototype is complete when ALL of the following are met:

#	Acceptance Criterion
AC-01	python run_agents.py executes end-to-end without exceptions on a clean environment with only .env populated
AC-02	scored_quant.json contains valid integer scores for all 6 Lynch quantitative metrics for all 8 stocks
AC-03	scored_kpi.json contains valid integer scores for all 5 IT KPI metrics for all 8 stocks
AC-04	report_YYYYMMDD.html opens in Chrome/Edge/Firefox and displays all 8 stocks ranked by Lynch Score
AC-05	Score bar charts render correctly — quant (blue) and KPI (green) bars visible for each stock
AC-06	Verdict badges (STRONG BUY / BUY / WATCH / NEUTRAL / AVOID) are correctly colour-coded
AC-07	The report is fully standalone — works without internet (except Chart.js CDN for charts)
AC-08	README.md documents setup steps, .env variables required, and how to run the project
AC-09	At least one Fast Grower (Persistent or Coforge) ranks higher than a Stalwart (TCS/Infosys) on Lynch Score
AC-10	3 ADK LlmAgent instances, @tool-decorated functions, and session-based output passing are demonstrably present

13. Phase 2 Roadmap

The following enhancements are planned after prototype submission, demonstrating the extensibility of the architecture:

Feature	Description	Data Source	Impact
Concall Transcript Agent	BSE filing poller downloads quarterly PDFs; LLM extracts key information	BSE Filing, Attention	High
Expanded Universe	Add 6 more mid/small-cap IT stocks; extend to Pharma/Financials sector	IndianAPI	Medium
Scheduled Runs	Daily cron job at 8AM IST; report emailed or sent to Slack	External pipeline	Medium
Score History (DuckDB)	Store Lynch scores over time; show trend arrows in report	Local DuckDB	Medium
MCP Server Externalisation	Publish IndianAPI tools as a proper MCP server endpoint	MCP protocol	High

Document end — Peter Lynch IT Stock Research Agent BRD v1.0
Built to demonstrate: Google ADK multi-agent orchestration · MCP tool integration · LLM-driven equity analysis